

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I Kamalli shree V(192525030) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.

CYBERSECURITY THREAT DETECTION USING LOGIC

1. Problem Analysis and Knowledge Identification

An AI-based cybersecurity system monitors User101. The system observes repeated failed login attempts, an unusual login location, and unauthorized access to confidential files. The objective is to determine whether a high-priority security alert should be generated and whether the activity indicates a possible security breach.

Entities

- User101
- Confidential files
- Login attempts
- Login location
- Security alert

Constants

- User101, ConfidentialFiles

Predicates

- User(x)
- FailedLogin(x)
- UnusualLocation(x)
- BruteForce(x)
- SuspiciousActivity(x)
- UnauthorizedAccess(x,y)
- SecurityBreach(x)
- SecurityAlert(x)
- HighPriorityAlert(x)
- SecurityMonitoring(x)

2. Knowledge Base

The following facts are assumed as observations available to the cybersecurity system:

Fact	Statement
F1	User(User101)
F2	FailedLogin(User101)
F3	FiveFailedLogins(User101)
F4	UnusualLocation(User101)
F5	UnauthorizedAccess(User101, ConfidentialFiles)
F6	ConfidentialFilesAccessed(User101)
F7	AccessOutsidePolicy(User101)
F8	LoginAttemptsHigh(User101)
F9	SecurityMonitoring(User101)
F10	AccountActive(User101)

The key facts required for the final inference are F3, F4, F5 and F9.

3. Eight Rules

R1 — Brute-Force Detection

- $\text{FiveFailedLogins}(x) \rightarrow \text{BruteForce}(x)$

R2 — Suspicious Login

- $\text{BruteForce}(x) \wedge \text{UnusualLocation}(x) \rightarrow \text{SuspiciousActivity}(x)$

R3 — Security Breach

- $\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x, \text{Confidential Files}) \rightarrow \text{SecurityBreach}(x)$

R4 — Security Alert

- $\text{SecurityBreach}(x) \rightarrow \text{SecurityAlert}(x)$

R5 — High Priority Alert

- $\text{SecurityAlert}(x) \wedge \text{AccessOutsidePolicy}(x) \rightarrow \text{HighPriorityAlert}(x)$

R6 — High Login Activity

- $\text{UnusualLocation}(x) \wedge \text{LoginAttemptsHigh}(x) \rightarrow \text{SuspiciousActivity}(x)$

R7 — Confidential File Alert

- $\text{SuspiciousActivity}(x) \wedge \text{ConfidentialFilesAccessed}(x) \rightarrow \text{SecurityAlert}(x)$

R8 — Monitored Security Breach

- $\text{SecurityBreach}(x) \wedge \text{SecurityMonitoring}(x) \rightarrow \text{HighPriorityAlert}(x)$

4. Propositional Logic Representation

For User101, define:

- F = Five failed login attempts
- U = Unusual location
- B = Brute-force attack
- S = Suspicious activity
- A = Unauthorized access
- R = Security breach
- M = Security monitoring
- H = High-priority alert

The main reasoning becomes:

- $F \rightarrow B$
- $B \wedge U \rightarrow S$
- $S \wedge A \rightarrow R$
- $R \wedge M \rightarrow H$

Given: F, U, A, M

Therefore: H

5. First-Order Logic Representation

Facts

- `FiveFailedLogins(User101)`
- `UnusualLocation(User101)`
- `UnauthorizedAccess(User101,ConfidentialFiles)`
- `SecurityMonitoring(User101)`

Rules

- $\forall x(\text{FiveFailedLogins}(x) \rightarrow \text{BruteForce}(x))$

- $\forall x(\text{BruteForce}(x) \wedge \text{UnusualLocation}(x) \rightarrow \text{SuspiciousActivity}(x))$
- $\forall x(\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x, \text{ConfidentialFiles}) \rightarrow \text{SecurityBreach}(x))$
- $\forall x(\text{SecurityBreach}(x) \wedge \text{SecurityMonitoring}(x) \rightarrow \text{HighPriorityAlert}(x))$

6. Unification and Substitution

Consider the rule:

- $\text{FiveFailedLogins}(x) \rightarrow \text{BruteForce}(x)$

and the fact:

- $\text{FiveFailedLogins}(\text{User101})$

Unification

Match:

- $\text{FiveFailedLogins}(x)$

with:

- $\text{FiveFailedLogins}(\text{User101})$

The most general unifier is:

$$\theta = \{x/\text{User101}\}$$

Substitution

Substitute $x = \text{User101}$:

- $\text{BruteForce}(x)$

becomes:

- $\text{BruteForce}(\text{User101})$

Similarly:

- $\text{BruteForce}(x) \wedge \text{UnusualLocation}(x)$

becomes:

- $\text{BruteForce}(\text{User101}) \wedge \text{UnusualLocation}(\text{User101})$

which produces:

$\text{SuspiciousActivity}(\text{User101})$

7. Forward Chaining

Forward chaining starts with the known facts and repeatedly applies rules to generate new facts.

Initial Facts

- F3=FiveFailedLogins(User101)
- F4=UnusualLocation(User101)
- F5=UnauthorizedAccess(User101,ConfidentialFiles)
- F9=SecurityMonitoring(User101)

Step 1 : Apply R1

- FiveFailedLogins(User101)

therefore:

- BruteForce(User101)

Step 2 : Apply R2

We now have:

- BruteForce(User101)

and:

- UnusualLocation(User101)

Therefore:

- SuspiciousActivity(User101)

Step 3 : Apply R3

We have:

- SuspiciousActivity(User101)

and:

- UnauthorizedAccess(User101,ConfidentialFiles)

Therefore:

- SecurityBreach(User101)

Step 4 — Apply R8

We have:

- SecurityBreach(User101)

and:

- SecurityMonitoring(User101)

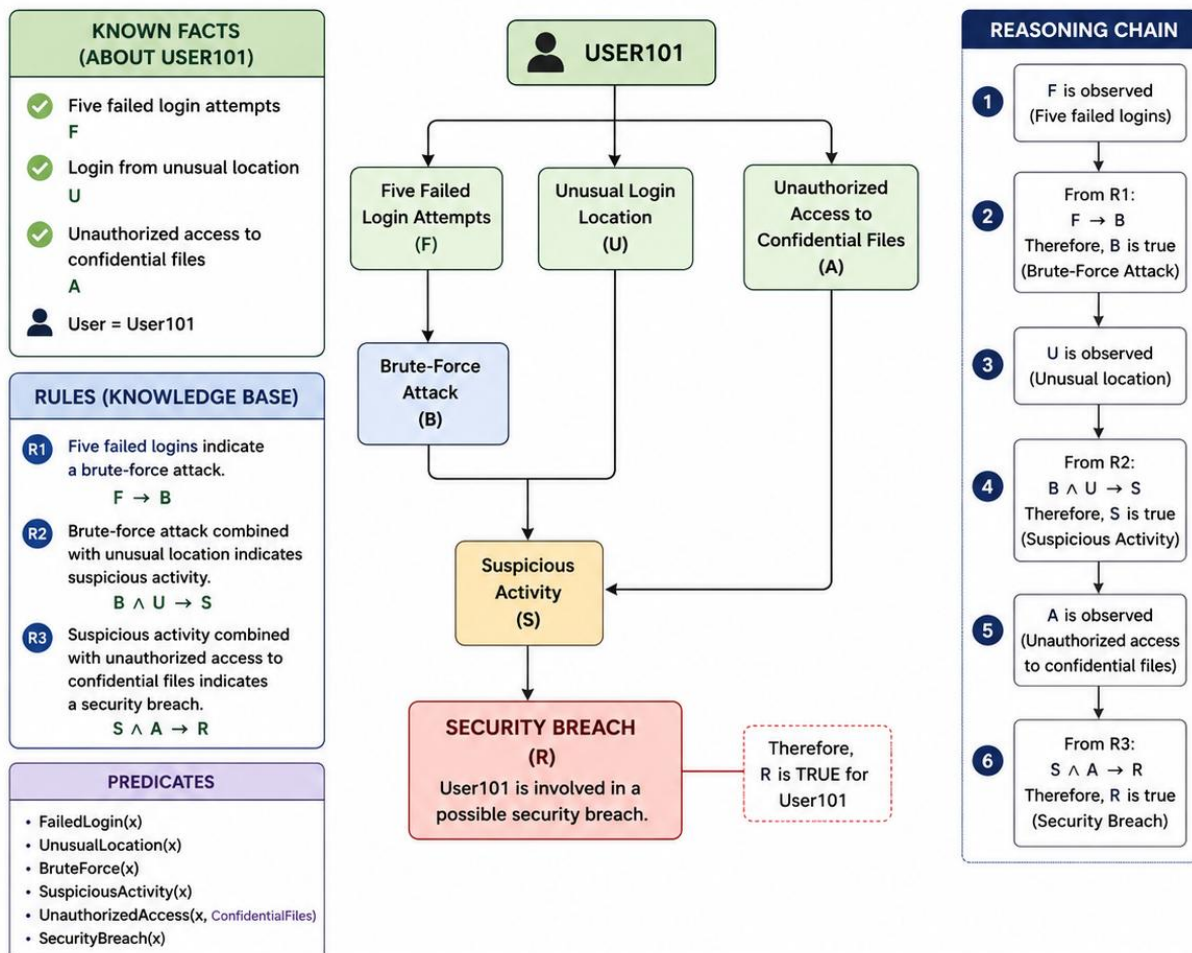
Therefore:

- HighPriorityAlert(User101)

Forward-Chaining Result

$F3 \rightarrow B \rightarrow S \rightarrow R \rightarrow H$

8. Forward Chaining and Reasoning Chain



9. Backward Chaining

The required query is:

- HighPriorityAlert(User101)

Backward chaining starts with this goal and works backwards to known facts.

Step 1

Goal:

- HighPriorityAlert(User101)

Which rule can produce it?

R8:

- $\text{SecurityBreach}(x) \wedge \text{SecurityMonitoring}(x) \rightarrow \text{HighPriorityAlert}(x)$

Therefore, we need:

- SecurityBreach(User101)

and

- SecurityMonitoring(User101)

Step 2

- $\text{SecurityMonitoring}(\text{User101})$ is already a known fact.

Now prove:

- $\text{SecurityBreach}(\text{User101})$

R3 gives:

- $\text{SuspiciousActivity}(x) \wedge \text{UnauthorizedAccess}(x, \text{Confidential Files}) \rightarrow \text{SecurityBreach}(x)$

Therefore, we need:

- $\text{SuspiciousActivity}(\text{User101})$

and:

- $\text{UnauthorizedAccess}(\text{User101}, \text{ConfidentialFiles})$

The second is already a known fact.

Step 3

To prove:

- $\text{SuspiciousActivity}(\text{User101})$

use R2:

- $\text{BruteForce}(x) \wedge \text{UnusualLocation}(x) \rightarrow \text{SuspiciousActivity}(x)$

We need:

- $\text{BruteForce}(\text{User101})$

and:

- $\text{UnusualLocation}(\text{User101})$

Unusual location is already known.

Step 4

To prove:

- $\text{BruteForce}(\text{User101})$

use R1:

- $\text{FiveFailedLogins}(x) \rightarrow \text{BruteForce}(x)$

We have:

- $\text{FiveFailedLogins}(\text{User101})$

Therefore:

- $\text{BruteForce}(\text{User101})$

Backward Chain

HighPriorityAlert←**SecurityBreach**←**SuspiciousActivity**←**Brute Force**←**FiveFailedLogins**

All required facts are available.

Therefore:

- HighPriorityAlert(User101)

10. Resolution Proof

To prove:

- HighPriorityAlert(User101)

we assume its negation:

- $\neg$ HighPriorityAlert(User101)

Convert relevant rules into clauses

R1: $\neg$ FVB

R2: $\neg$ BV $\neg$ UVS

R3: $\neg$ SV $\neg$ AVR

R8: $\neg RV \neg MVH$

Negated goal: $\neg H$

Facts: F, U, A, M

Resolution Step 1

From: F

and: $\neg FVB$

derive: **B**

Resolution Step 2

From: B

and: $\neg BV \neg UVS$

derive: $\neg UVS$

Using U: **S**

Resolution Step 3

From: S

and: $\neg SV \neg AVR$

derive: $\neg AVR$

R

Using A:

Resolution Step 4

From: R

and: $\neg R \vee \neg M \vee H$

derive: $\neg M \vee H$

Using M:

H

Resolution Step 5

We have: H

and the negated query: $\neg H$

Therefore: $H \wedge \neg H$

produces:

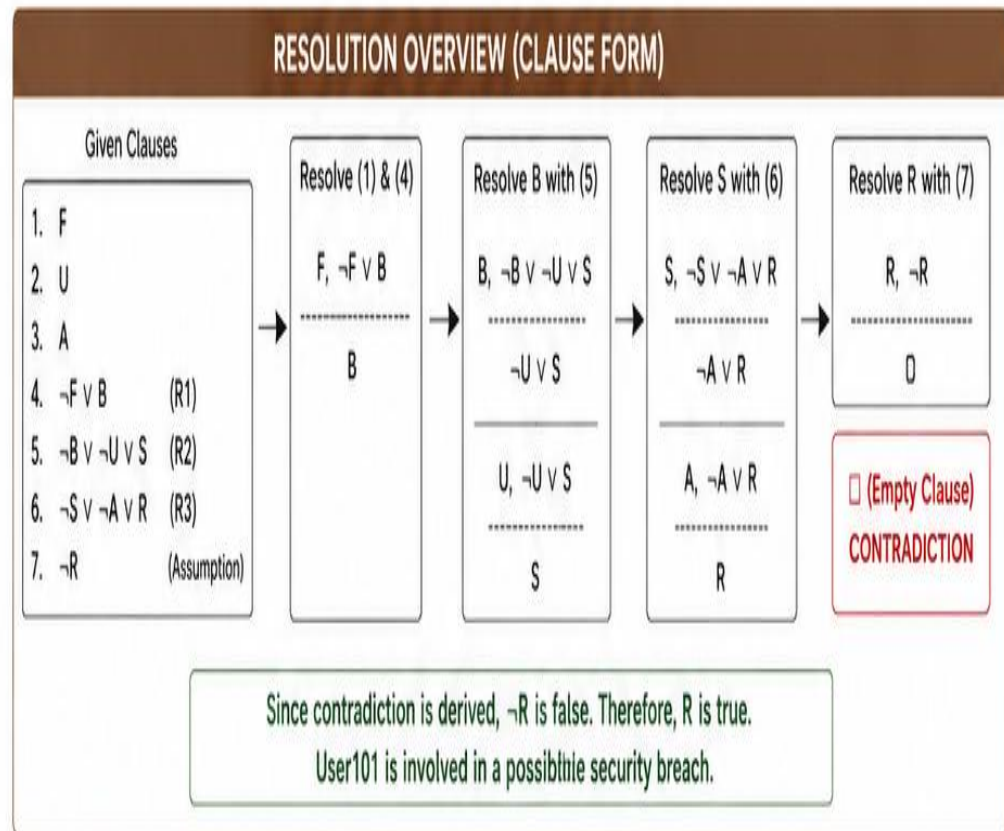
The empty clause represents a contradiction.

Therefore:

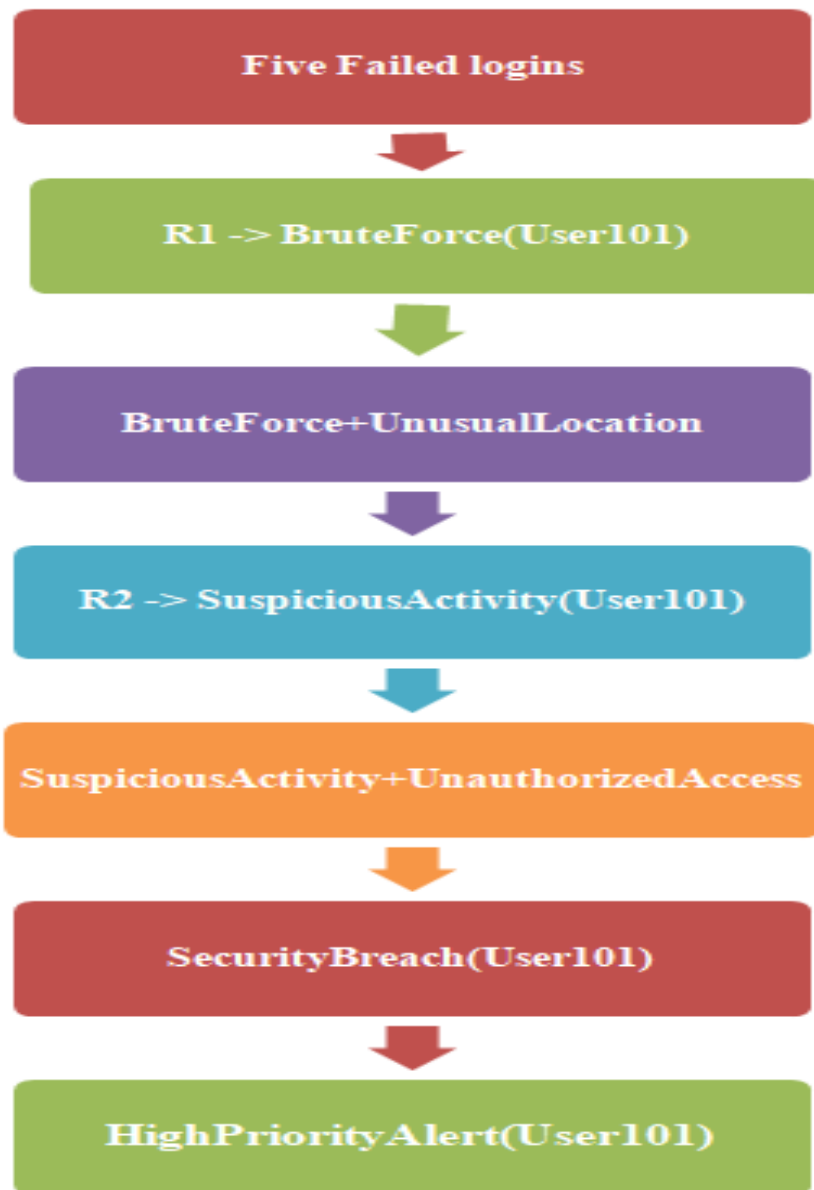
- HighPriorityAlert(User101) is proven.

11. Resolution Proof

SYMBOL LEGEND	
F	: Five failed login attempts
U	: Unusual login location
B	: Brute-force attack
S	: Suspicious activity
A	: Unauthorized access
R	: Security breach



12. Complete Reasoning Chain



13. Forward vs Backward Chaining

Forward Chaining	Backward Chaining
Starts with known facts	Starts with a goal
Data-driven	Goal-driven
Applies rules to derive new facts	Finds rules that can prove the goal
May generate irrelevant facts	Focuses on relevant rules
Suitable for continuous monitoring	Suitable for investigating a specific alert
Can generate multiple conclusions	Usually explores one target conclusion

Which is more suitable?

For real-time cybersecurity monitoring, Forward Chaining is useful because new security events can continuously trigger rules and generate alerts.

For investigating a particular question such as HighPriorityAlert(User101) Backward Chaining is more efficient because it starts with the specific alert and checks only the rules and facts required to prove it.

Therefore, a practical cybersecurity system can use both:

Forward Chaining for monitoring + Backward Chaining for investigation

14. Advantages

- Provides transparent, rule-based threat reasoning.
- Every conclusion can be traced back to known facts.
- FOL allows rules to be generalized to multiple users.
- Forward chaining is useful for continuous security monitoring.
- Backward chaining efficiently investigates specific security alerts.

15. Limitations

- Accuracy depends on the completeness of the knowledge base.
- New attack patterns require new rules.
- Basic logic does not naturally represent uncertainty.
- Large knowledge bases can increase inference complexity.
- False or incomplete observations can lead to incorrect conclusions.

16. Conclusion

The cybersecurity threat-detection model represents security observations as facts and uses logical rules to derive increasingly significant security conditions. Starting with the observed five failed login attempts, the system derives a brute-force attack. When the unusual login location is considered, the activity is classified as suspicious. The combination of suspicious activity and unauthorized access to confidential files establishes a possible security breach. Because the security

system is monitoring the account, the reasoning ultimately produces the query result `HighPriorityAlert(User101)`.

The same conclusion can be reached through both forward and backward reasoning. Forward chaining begins with the observed security events and progressively derives new facts, making it suitable for continuous monitoring. Backward chaining begins with the specific alert query and works backward to determine whether sufficient evidence exists. Resolution provides an additional formal proof by deriving a contradiction from the negated query.

Thus, the knowledge representation model demonstrates how Propositional Logic, First-Order Logic, unification, substitution, forward chaining, backward chaining, and resolution can work together to provide an explainable cybersecurity inference system. Within the assumptions encoded in the knowledge base, `User101` should be flagged with a high-priority security alert and considered involved in a possible security breach.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.